UD3 - Consultas SQL (IV)

Nesta parte da unidade didáctica estudaremos as subconsultas, que permiten utilizar o resultado dunha consulta SQL dentro doutra consulta. Aprenderemos a utilizar subconsultas escalares, operadores de pertenza, condicións de existencia, funcións de resumo e subconsultas correlacionadas.

18. Introdución ás subconsultas

18.1. Que é unha subconsulta?

Unha subconsulta é unha consulta SQL incluída dentro doutra instrución SQL. A consulta interior proporciona un resultado que pode ser utilizado pola consulta principal para filtrar, comparar ou seleccionar información.

As subconsultas escríbense normalmente entre parénteses e poden aparecer en diferentes partes dunha instrución `SELECT`, especialmente nas cláusulas `WHERE`, `HAVING` e `FROM`.

```
SELECT columnas
FROM taboa
WHERE columna operador (
    SELECT columna
    FROM outra_taboa
    WHERE condicion
);
```

A consulta interior proporciona o valor ou os valores que necesita a consulta exterior.

> **Idea clave:** unha subconsulta permite resolver unha condición utilizando información que se obtén doutra consulta, sen necesidade de realizar varias consultas independentes.

18.2. Orde de execución

Nunha subconsulta non correlacionada, a consulta interior pode executarse de maneira independente da consulta principal. Conceptualmente, primeiro obtense o resultado da subconsulta e despois utilízase na consulta exterior.

Esta é unha forma útil de entender as subconsultas, aínda que o optimizador de MariaDB pode transformar a consulta e executar ambas as partes mediante outro plan de execución.

18.3. Subconsultas escalares

Unha subconsulta escalar é aquela que devolve un único valor: unha fila e unha columna.

Este valor pode utilizarse nunha comparación mediante operadores como:

- `=` : igual.
- `<>` : diferente.
- `>` : maior que.
- `<` : menor que.
- `>=` : maior ou igual que.
- `<=` : menor ou igual que.

18.4. Exemplo con operador de igualdade

Supoñamos que queremos obter os alumnos que pertencen ao curso DAW1.

Podemos utilizar unha subconsulta para obter primeiro o identificador do curso DAW1:

```
SELECT id
FROM cursos
WHERE nombre = 'DAW1';
```

O resultado será o identificador dese curso. Podemos utilizar directamente esta consulta dentro doutra:

```
SELECT nombre
FROM alumnos
WHERE curso_id = (
    SELECT id
    FROM cursos
    WHERE nombre = 'DAW1'
);
```

A subconsulta obtén o identificador do curso DAW1 e a consulta principal selecciona os alumnos que teñen ese identificador no campo `curso_id`.

18.5. Subconsultas con operadores de comparación

Podemos utilizar operadores de comparación para relacionar os datos dunha táboa cun valor calculado mediante unha subconsulta.

Por exemplo, queremos obter os alumnos que teñen unha idade superior á idade media de todos os alumnos.

```
SELECT nombre, edad
FROM alumnos
WHERE edad > (
    SELECT AVG(edad)
    FROM alumnos
);
```

A subconsulta calcula a idade media e a consulta principal mostra os alumnos que superan ese valor.

18.6. Comparar cun valor máximo ou mínimo

As funcións de resumo tamén poden utilizarse dentro das subconsultas escalares.

Obter os alumnos que teñen a idade máxima:

```
SELECT nombre, edad
FROM alumnos
WHERE edad = (
    SELECT MAX(edad)
    FROM alumnos
);
```

Obter os alumnos que teñen unha idade inferior á idade máxima:

```
SELECT nombre, edad
FROM alumnos
WHERE edad < (
    SELECT MAX(edad)
    FROM alumnos
);
```

18.7. Consideracións importantes

- Unha subconsulta escalar debe devolver como máximo unha fila e unha columna.
- Se devolve máis dunha fila, unha comparación como `=` produce un erro.
- Se non devolve ningunha fila, o resultado escalar será `NULL`.
- Se a subconsulta devolve `NULL`, as comparacións habituais non serán verdadeiras nin falsas, senón descoñecidas.

> **Atención:** se unha subconsulta pode devolver varios valores, non debemos utilizar directamente operadores como `=` ou `>`. Nese caso podemos necesitar `IN`, `EXISTS` ou outros operadores específicos.

18.8. Exercicios de consolidación

1. Obtén os alumnos que pertencen ao curso DAW2 mediante unha subconsulta.
2. Mostra os alumnos que teñen unha idade superior á media.
3. Obtén os alumnos que teñen a idade mínima.
4. Mostra os cursos que teñen un número de horas superior á media de horas dos cursos.
5. Obtén os alumnos que pertencen ao curso que ten o maior número de horas.

19. Subconsultas con IN e NOT IN

19.1. Subconsultas que devolven varios valores

En ocasións, unha subconsulta pode devolver varios rexistros. Por exemplo, podemos necesitar obter todos os identificadores dos cursos que cumpren unha determinada condición.

Para utilizar estes resultados na consulta principal podemos empregar os operadores `IN` e `NOT IN`.

19.2. Operador IN

O operador `IN` comproba se un valor pertence ao conxunto de valores devolto pola subconsulta.

```
SELECT columnas
FROM taboa
WHERE columna IN (
    SELECT columna
    FROM outra_taboa
    WHERE condicion
);
```

19.3. Exemplo: alumnos pertencentes a determinados cursos

Supoñamos que queremos obter os alumnos que pertencen a cursos cunha duración superior a 180 horas.

Primeiro podemos obter os identificadores dos cursos que cumpren esa condición:

```
SELECT id
FROM cursos
WHERE horas > 180;
```

Despois utilizamos a consulta anterior como subconsulta:

```
SELECT nombre, curso_id
FROM alumnos
WHERE curso_id IN (
    SELECT id
    FROM cursos
    WHERE horas > 180
);
```

A consulta principal mostra os alumnos cuxo identificador de curso aparece no resultado da subconsulta.

19.4. Operador NOT IN

O operador `NOT IN` permite seleccionar os rexistros cuxo valor non aparece no conxunto devolto pola subconsulta.

```
SELECT columnas
FROM taboa
WHERE columna NOT IN (
    SELECT columna
    FROM outra_taboa
    WHERE condicion
);
```

19.5. Exemplo: alumnos que non pertencen a cursos longos

Queremos obter os alumnos que non pertencen a cursos cunha duración superior a 180 horas.

```
SELECT nombre, curso_id
FROM alumnos
WHERE curso_id NOT IN (
    SELECT id
    FROM cursos
    WHERE horas > 180
);
```

A subconsulta devolve os identificadores dos cursos con máis de 180 horas. A consulta principal selecciona os alumnos cuxo curso non aparece nese conxunto.

19.6. Diferenza entre IN e NOT IN

Operador	Funcionamento
`IN`	Selecciona os rexistros cuxo valor aparece no resultado da subconsulta.
`NOT IN`	Selecciona os rexistros cuxo valor non aparece no resultado da subconsulta.

19.7. Atención aos valores NULL con NOT IN

Hai que ter especial coidado cando a subconsulta pode devolver valores `NULL`.

Por exemplo:

```
SELECT nombre
FROM alumnos
WHERE curso_id NOT IN (
    SELECT id
    FROM cursos
);
```

Se o resultado da subconsulta contén un valor `NULL`, as comparacións de `NOT IN` poden producir un resultado descoñecido e impedir que se devolvan os rexistros esperados.

Para evitar este problema, podemos excluír explicitamente os valores nulos na subconsulta:

```
SELECT nombre
FROM alumnos
WHERE curso_id NOT IN (
    SELECT id
    FROM cursos
    WHERE id IS NOT NULL
);
```

> **Recomendación:** cando utilices `NOT IN`, comproba que a subconsulta non pode devolver valores `NULL`. Para detectar rexistros sen correspondencia, tamén podes utilizar `NOT EXISTS`.

19.8. Exercicios de consolidación

1. Obtén os alumnos que pertencen a cursos con máis de 150 horas.
2. Mostra os alumnos que non pertencen a cursos con máis de 180 horas.
3. Obtén os cursos que teñen alumnos maiores de idade.
4. Mostra os cursos que non teñen alumnos menores de idade.
5. Obtén os alumnos que pertencen a cursos cuxo número de horas é igual ou superior a 190.

20. Subconsultas con EXISTS e NOT EXISTS

20.1. Que é EXISTS?

O operador `EXISTS` permite comprobar se unha subconsulta devolve polo menos un rexistro.

Non importa o contido das filas devoltas pola subconsulta: o que interesa é saber se existe algún resultado.

```
SELECT columnas
FROM taboa_principal
WHERE EXISTS (
    SELECT *
    FROM outra_taboa
    WHERE condicion
);
```

Se a subconsulta devolve polo menos unha fila, `EXISTS` é verdadeiro e o rexistro da consulta principal pode formar parte do resultado.

20.2. Exemplo: cursos que teñen alumnos

Queremos obter os cursos que teñen polo menos un alumno asociado.

```
SELECT c.id,
       c.nombre
FROM cursos c
WHERE EXISTS (
    SELECT *
    FROM alumnos a
    WHERE a.curso_id = c.id
);
```

Para cada curso, a subconsulta comproba se existe algún alumno cuxo `curso_id` coincida co identificador do curso.

20.3. Que é NOT EXISTS?

O operador `NOT EXISTS` realiza a comprobación contraria: é verdadeiro cando a subconsulta non devolve ningún rexistro.

```
SELECT columnas
FROM taboa_principal
WHERE NOT EXISTS (
    SELECT *
    FROM outra_taboa
    WHERE condicion
);
```

20.4. Exemplo: cursos sen alumnos

Queremos obter os cursos que non teñen ningún alumno asociado.

```
SELECT c.id,
       c.nombre
FROM cursos c
WHERE NOT EXISTS (
    SELECT *
    FROM alumnos a
    WHERE a.curso_id = c.id
);
```

Para cada curso, a subconsulta busca alumnos relacionados. Se non atopa ningún, o curso aparece no resultado.

20.5. EXISTS fronte a IN

Tanto `IN` como `EXISTS` permiten filtrar rexistros a partir de información doutra consulta, pero funcionan de maneira diferente.

Exemplo utilizando `IN`:

```
SELECT nombre
FROM alumnos
WHERE curso_id IN (
    SELECT id
    FROM cursos
    WHERE horas > 180
);
```

A subconsulta devolve un conxunto de identificadores e `IN` comproba se o identificador do alumno pertence a ese conxunto.

A mesma idea pode expresarse con `EXISTS`:

```
SELECT a.nombre
FROM alumnos a
WHERE EXISTS (
    SELECT *
    FROM cursos c
    WHERE c.id = a.curso_id
      AND c.horas > 180
);
```

A segunda consulta comproba se existe un curso relacionado co alumno que cumpra a condición indicada.

20.6. EXISTS e NOT EXISTS con varias condicións

Podemos utilizar varias condicións dentro da subconsulta para comprobar a existencia de rexistros que cumpran determinados requisitos.

Por exemplo, obter os alumnos que teñen polo menos unha matrícula cunha nota igual ou superior a 5:

```
SELECT a.id,
       a.nombre
FROM Alumno a
WHERE EXISTS (
    SELECT *
    FROM Matricula ma
    WHERE ma.id_alumno = a.id
      AND ma.nota >= 5
);
```

Para cada alumno compróbase se existe algunha matrícula cunha nota igual ou superior a 5.

Se queremos obter os alumnos que non teñen ningunha matrícula aprobada, podemos utilizar:

```
SELECT a.id,
       a.nombre
FROM Alumno a
WHERE NOT EXISTS (
    SELECT *
    FROM Matricula ma
    WHERE ma.id_alumno = a.id
      AND ma.nota >= 5
);
```

20.7. Exercicios de consolidación

1. Obtén os cursos que teñen polo menos un alumno.
2. Obtén os cursos que non teñen ningún alumno.
3. Mostra os alumnos que teñen polo menos unha matrícula.
4. Mostra os alumnos que non teñen ningunha matrícula.
5. Obtén os alumnos que teñen polo menos unha matrícula aprobada.
6. Obtén os alumnos que non teñen ningunha matrícula aprobada.

21. Subconsultas con funcións de resumo e agrupamentos

21.1. Utilizar funcións de resumo nas subconsultas

As subconsultas poden utilizar funcións de resumo como `COUNT()`, `SUM()`, `AVG()`, `MIN()` e `MAX()`.

Estas funcións permiten calcular valores que posteriormente poden ser utilizados pola consulta principal.

21.2. Comparar cunha media

Queremos obter os alumnos cuxa idade é superior á idade media de todos os alumnos.

```
SELECT nombre,
       edad
FROM alumnos
WHERE edad > (
    SELECT AVG(edad)
    FROM alumnos
);
```

A subconsulta calcula unha única media e a consulta principal utiliza ese valor para seleccionar os alumnos.

21.3. Comparar co número máximo de horas

Podemos obter os cursos que teñen o maior número de horas mediante unha subconsulta:

```
SELECT nombre,
       horas
FROM cursos
WHERE horas = (
    SELECT MAX(horas)
    FROM cursos
);
```

A subconsulta calcula o número máximo de horas e a consulta principal selecciona os cursos que teñen ese valor.

21.4. Subconsultas con GROUP BY

Tamén podemos realizar agrupamentos dentro dunha subconsulta. Isto permite calcular valores para cada grupo e utilizar posteriormente eses resultados na consulta principal.

Por exemplo, queremos obter os identificadores dos cursos que teñen máis de dous alumnos.

```
SELECT curso_id
FROM alumnos
GROUP BY curso_id
HAVING COUNT(*) > 2;
```

A consulta agrupa os alumnos por curso e selecciona os grupos que teñen máis de dous alumnos.

Podemos utilizar esta consulta como subconsulta:

```
SELECT id,
       nombre
FROM cursos
WHERE id IN (
    SELECT curso_id
    FROM alumnos
    GROUP BY curso_id
    HAVING COUNT(*) > 2
);
```

A consulta principal mostra os cursos que teñen máis de dous alumnos.

21.5. Subconsultas con SUM

Supoñamos que queremos obter os módulos nos que a suma das notas das matrículas é superior a un determinado valor.

Podemos calcular primeiro os módulos que cumpren esa condición:

```
SELECT id_modulo
FROM Matricula
GROUP BY id_modulo
HAVING SUM(nota) > 20;
```

A consulta principal utiliza os identificadores obtidos:

```
SELECT id,
       nombre
FROM Modulo
WHERE id IN (
    SELECT id_modulo
    FROM Matricula
    GROUP BY id_modulo
    HAVING SUM(nota) > 20
);
```

21.6. Subconsultas con AVG e agrupamentos

Podemos calcular a nota media de cada módulo e utilizar os resultados para seleccionar aqueles módulos cuxa media supera un valor determinado.

```
SELECT id,
       nombre
FROM Modulo
WHERE id IN (
    SELECT id_modulo
    FROM Matricula
    GROUP BY id_modulo
    HAVING AVG(nota) >= 7
);
```

A subconsulta calcula a media das notas por módulo e selecciona os identificadores dos módulos cunha media igual ou superior a 7.

21.7. Combinar subconsultas con consultas principais

Unha subconsulta pode formar parte dunha consulta principal que tamén conteña composicións entre táboas, filtros ou ordenacións.

Por exemplo, queremos mostrar os nomes dos alumnos que teñen unha nota superior á media de todas as matrículas:

```
SELECT a.nombre,
       ma.nota
FROM Alumno a
INNER JOIN Matricula ma
    ON a.id = ma.id_alumno
WHERE ma.nota > (
    SELECT AVG(nota)
    FROM Matricula
)
ORDER BY ma.nota DESC;
```

A subconsulta calcula a nota media global e a consulta principal mostra as matrículas que superan esa media, xunto co nome dos alumnos.

21.8. Exercicios de consolidación

1. Obtén os cursos que teñen máis de dous alumnos mediante unha subconsulta con `GROUP BY` e `HAVING`.
2. Mostra os cursos que teñen un número de horas superior á media de todos os cursos.
3. Obtén os módulos nos que a nota media das matrículas é igual ou superior a 7.
4. Mostra os alumnos que teñen algunha nota superior á media global das matrículas.
5. Obtén os módulos que teñen máis de cinco matrículas.
6. Mostra os cursos cuxo número de alumnos é superior á media de alumnos por curso.

22. Subconsultas correlacionadas

22.1. Que é unha subconsulta correlacionada?

Unha subconsulta correlacionada é aquela que utiliza información da consulta principal.

A diferenza dunha subconsulta independente, a subconsulta correlacionada fai referencia a unha columna da consulta exterior. Por este motivo, o seu resultado depende do rexistro que se está avaliando na consulta principal.

```
SELECT columnas
FROM taboa principal
WHERE condicion (
    SELECT *
    FROM outra_taboa
    WHERE outra_taboa.clave = principal.clave
);
```

> **Idea clave:** a subconsulta correlacionada utiliza valores do rexistro actual da consulta principal para realizar a súa comprobación.

22.2. Exemplo: cursos que teñen alumnos

Queremos obter os cursos que teñen polo menos un alumno. Podemos resolver este problema mediante unha subconsulta correlacionada con `EXISTS`.

```
SELECT c.id,
       c.nombre
FROM cursos c
WHERE EXISTS (
    SELECT *
    FROM alumnos a
    WHERE a.curso_id = c.id
);
```

A condición `a.curso_id = c.id` relaciona a subconsulta co rexistro actual da consulta principal.

Para cada curso, a subconsulta comproba se existe algún alumno asociado.

22.3. Exemplo: cursos sen alumnos

Se queremos obter os cursos que non teñen alumnos, podemos utilizar `NOT EXISTS`:

```
SELECT c.id,
       c.nombre
FROM cursos c
WHERE NOT EXISTS (
    SELECT *
    FROM alumnos a
    WHERE a.curso_id = c.id
);
```

22.4. Exemplo: alumnos cunha nota superior á súa media

Imos utilizar as táboas `Alumno` e `Matricula`.

Queremos obter as matrículas nas que a nota é superior á media das notas do mesmo alumno.

```
SELECT a.nombre,
       ma.nota
FROM Alumno a
INNER JOIN Matricula ma
    ON a.id = ma.id_alumno
WHERE ma.nota > (
    SELECT AVG(ma2.nota)
    FROM Matricula ma2
    WHERE ma2.id_alumno = a.id
);
```

A subconsulta calcula a nota media das matrículas do alumno actual. A consulta principal compara cada nota individual con esa media.

Os alias permiten distinguir as dúas referencias á táboa `Matricula`:

- `ma`: matrícula que se está avaliando na consulta principal.
- `ma2`: matrículas utilizadas para calcular a media do alumno.

22.5. Exemplo: alumnos que teñen todas as matrículas aprobadas

Queremos obter os alumnos que non teñen ningunha matrícula cunha nota inferior a 5.

```
SELECT a.id,
       a.nombre
FROM Alumno a
WHERE NOT EXISTS (
    SELECT *
    FROM Matricula ma
    WHERE ma.id_alumno = a.id
      AND ma.nota < 5
);
```

A subconsulta busca matrículas suspensas para cada alumno. Se non atopa ningunha, o alumno aparece no resultado.

> **Importante:** esta consulta tamén inclúe os alumnos que non teñen ningunha matrícula, porque tampouco existe para eles ningunha matrícula suspensa. Se queremos esixir que teñan polo menos unha matrícula, debemos engadir unha segunda condición `EXISTS`.

```
SELECT a.id,
       a.nombre
FROM Alumno a
WHERE EXISTS (
    SELECT *
    FROM Matricula ma
    WHERE ma.id_alumno = a.id
)
AND NOT EXISTS (
    SELECT *
    FROM Matricula ma
    WHERE ma.id_alumno = a.id
      AND ma.nota < 5
);
```

22.6. Diferenza entre subconsultas independentes e correlacionadas

Tipo	Características
Independente	Non depende de valores da consulta principal. Pode calcularse de maneira independente.
Correlacionada	Utiliza columnas da consulta principal e o seu resultado depende do rexistro que se está avaliando.

22.7. Consideracións sobre o rendemento

As subconsultas correlacionadas poden resultar máis complexas de executar, xa que dependen dos rexistros da consulta principal.

Non obstante, o optimizador de MariaDB pode transformar determinadas subconsultas para mellorar o seu rendemento. Non debemos asumir que unha subconsulta correlacionada se executa sempre literalmente unha vez por cada rexistro.

Para analizar o comportamento dunha consulta podemos utilizar `EXPLAIN`, que permite consultar o plan de execución.

```
EXPLAIN
SELECT c.id,
       c.nombre
FROM cursos c
WHERE EXISTS (
    SELECT *
    FROM alumnos a
    WHERE a.curso_id = c.id
);
```

22.8. Exercicios de consolidación

1. Obtén os cursos que teñen polo menos un alumno utilizando `EXISTS`.
2. Obtén os cursos que non teñen alumnos mediante `NOT EXISTS`.
3. Mostra os alumnos que teñen unha nota superior á súa propia media.
4. Obtén os alumnos que teñen todas as súas matrículas aprobadas e polo menos unha matrícula.
5. Mostra os módulos nos que existe algunha matrícula cunha nota inferior a 5.
6. Obtén os módulos nos que todas as matrículas teñen unha nota igual ou superior a 5, esixindo que exista polo menos unha matrícula.

Resumo das subconsultas

Tipo	Uso principal	Exemplo
Escalar	Comparar cun único valor.	`WHERE idade > (SELECT AVG(idade) ...)`
IN	Comprobar se un valor pertence a un conxunto.	`WHERE curso_id IN (SELECT ...)`
NOT IN	Comprobar se un valor non pertence a un conxunto.	`WHERE curso_id NOT IN (SELECT ...)`
EXISTS	Comprobar se existe polo menos un rexistro.	`WHERE EXISTS (SELECT ...)`
NOT EXISTS	Comprobar se non existe ningún rexistro.	`WHERE NOT EXISTS (SELECT ...)`
Correlacionada	Relacionar a subconsulta coa fila actual da consulta principal.	`WHERE ma.id_alumno = a.id`

As subconsultas permiten expresar condicións complexas utilizando información procedente doutras consultas. A elección do operador depende do tipo de resultado que devolve a subconsulta e da condición que queremos comprobar.